



Introduction to Algorithms

Step-by-Step Problem Solving for Computers

What is an Algorithm?



A step-by-step set of instructions to solve a specific problem or complete a task. Every algorithm has an input, a process, and an output.



GPS Directions



A Cooking Recipe



Looking Up a Word

INPUT

The problem or data



PROCESS

Step-by-step logic



OUTPUT

The solution or result

Algorithms in Everyday Life



Google Search

PageRank sorts billions of pages to show you the best results in milliseconds



Netflix / Spotify

Recommendation algorithms predict what you'll enjoy based on your history



GPS Navigation

Dijkstra's / A* algorithm finds the shortest path between two points



Social Media Feed

Ranking algorithms decide which posts appear first in your timeline



Online Shopping

Sorting algorithms rank products by price, reviews, or relevance



Uber / Lyft

Matching algorithms pair riders with the nearest available driver

You interact with hundreds of algorithms every day — most of them invisible.

Properties of a Good Algorithm

1



Clear Inputs & Outputs

Knows exactly what goes in and what comes out.

2



Well-Defined Steps

Each step is precise and unambiguous — no guessing.

3



Finite

It must eventually stop. An algorithm that runs forever is broken.

4



Effective

Every step must be doable and contribute to the solution.

5



Correct

Produces the right answer for all valid inputs — not just some.

If it's missing any of these — it's not an algorithm, it's just a rough idea.

Common Types of Algorithms



Searching

Find a specific item in a collection.

Examples: Linear Search, Binary Search



Sorting

Arrange items in a specific order.

Examples: Bubble Sort, Merge Sort, Quick Sort



Graph / Path

Find the best route between points.

Examples: Dijkstra's, BFS, DFS



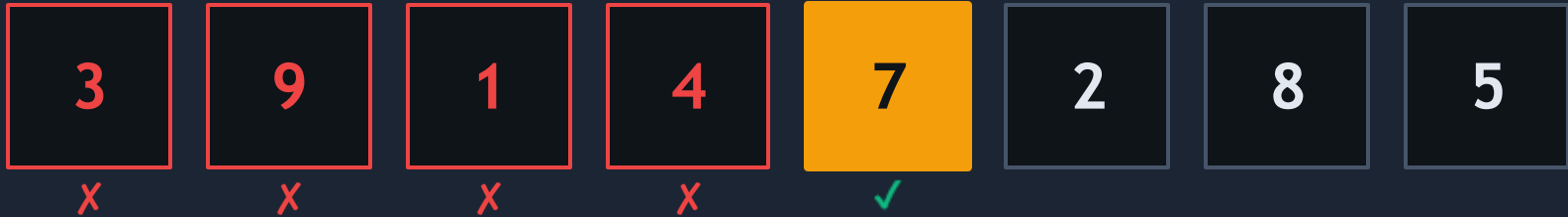
Divide & Conquer

Break a problem into smaller pieces, solve each, combine results.

Linear Search

Check every element one by one until you find the target.

Finding the number 7 in this list:



Checked 5 elements to find the target (index 4)



Best Case: $O(1)$

Target is the first element



Worst Case: $O(n)$

Target is the last element or missing

Binary Search

Requires sorted data. Cut the search space in half each step.

Finding the number 7 in a sorted list:

Step 1: Check middle → 4 (too small, go right)



Step 2: Check middle → 6 (too small, go right)



Step 3: Check middle → 7 ✓ Found!



Only 3 steps instead of 5! Worst case: $O(\log n)$ — for 1 billion items, just ~30 checks.

Sorting: Bubble Sort

Compare adjacent pairs — swap if out of order. Repeat until sorted.

Start:

5

3

8

1

4



Pass 1:

3

5

1

4

8



Pass 2:

3

1

4

5

8



Pass 3:

1

3

4

5

8

Largest values "bubble" to the end each pass

Time: $O(n^2)$

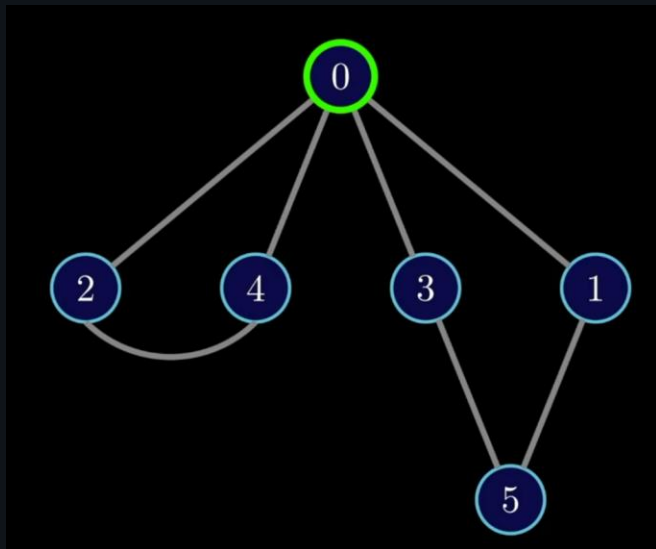
Slow for large lists

Easy to understand

Great for learning!

BFS: Breadth-First Search

Explores level by level — like a ripple spreading outward from the start.



Explore all neighbors before going deeper



Guarantees shortest path



Uses a Queue (FIFO)



Time: $O(V + E)$



Great for: mazes, social networks, GPS

Think of it like dropping a stone in water — the ripple checks every node at the same distance before moving further.

Big O Notation

How we measure how fast or slow an algorithm is as the input grows.

Big O	Name	Example	Speed	
$O(1)$	Constant	Accessing array[5]	Instant	
$O(\log n)$	Logarithmic	Binary Search	Very Fast	
$O(n)$	Linear	Linear Search	Fair	
$O(n \log n)$	Linearithmic	Merge Sort	Decent	
$O(n^2)$	Quadratic	Bubble Sort	Slow	
$O(2^n)$	Exponential	Brute-force subsets	Terrible	

Big O doesn't measure exact time — it measures how the time grows as the input gets bigger.

Operations

Horrible Bad Fair Good Excellent

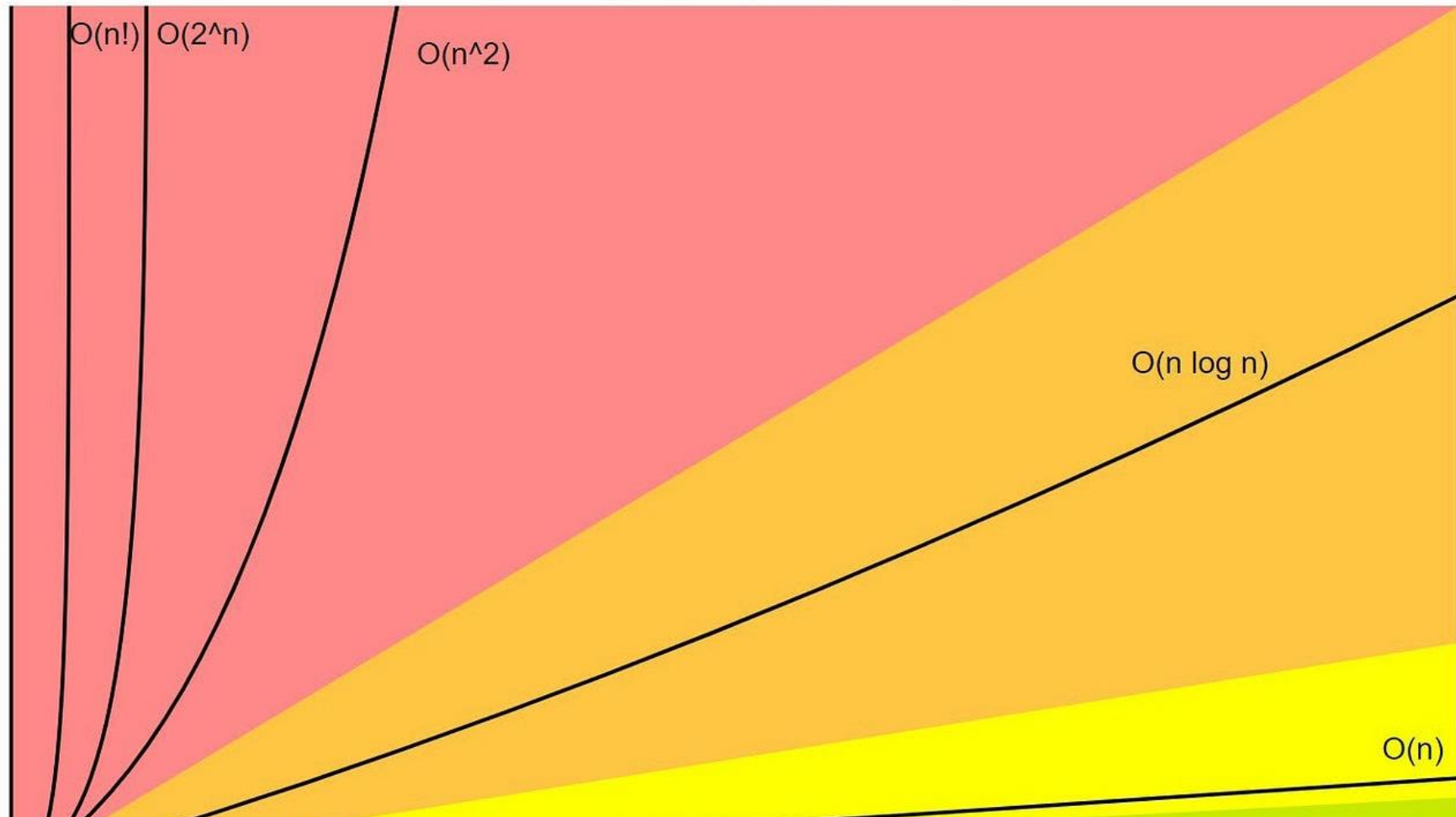
$O(n!)$

$O(2^n)$

$O(n^2)$

$O(n \log n)$

$O(n)$



Why Efficiency Matters

How many steps to find an item?

Items	Linear Search $O(n)$	Binary Search $O(\log n)$
10	10 steps	4 steps
1,000	1,000 steps	10 steps
1,000,000	1,000,000 steps	20 steps
1 Billion	1,000,000,000 steps	30 steps



A billion items: Linear takes forever. Binary takes 30 steps. The right algorithm changes everything.

Writing Pseudocode

Pseudocode is plain-English logic — not real code, but structured enough to follow.

Linear Search

```
FUNCTION linearSearch(list, target):
```

```
  FOR each item in list:
```

```
    IF item equals target:
```

```
      RETURN item's position
```

```
  RETURN "not found"
```

Binary Search

```
FUNCTION binarySearch(list, target):
```

```
  SET low = 0, high = length - 1
```

```
  WHILE low <= high:
```

```
    SET mid = (low + high) / 2
```

```
    IF list[mid] = target: RETURN mid
```

```
    IF list[mid] < target: low = mid+1
```

```
    ELSE: high = mid - 1
```

```
  RETURN "not found"
```

Pseudocode lets you plan your logic before worrying about syntax.

Key Takeaways



Algorithms are step-by-step instructions to solve problems



Searching & sorting are the building blocks of computer science



Big O measures efficiency — the right algorithm can be millions of times faster



Pseudocode helps you plan before you code

Think like a computer: break problems into steps, measure efficiency, optimize!